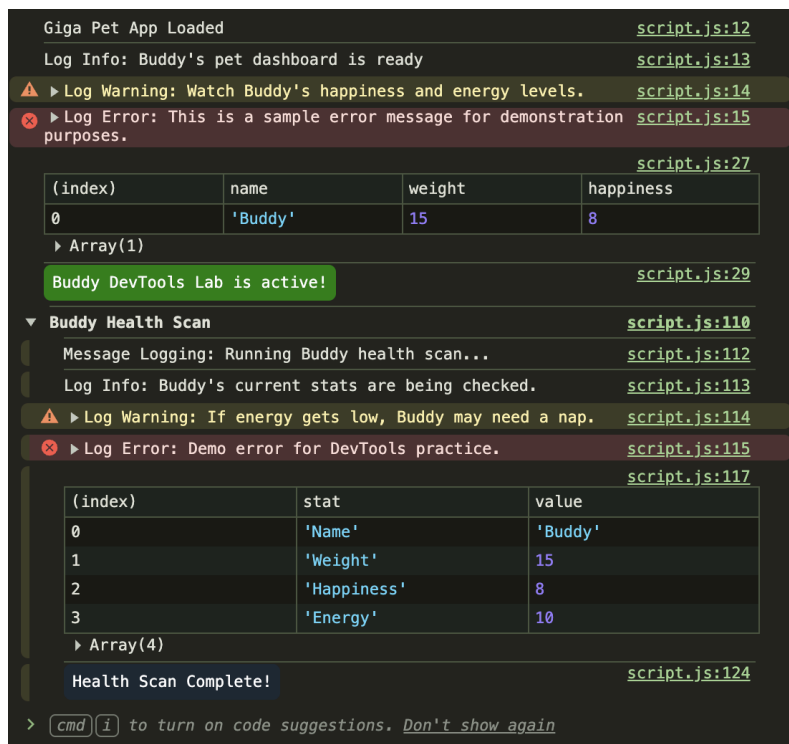


Chrome DevTools Implementations

1. Health Scan Logging System

For this implementation, I added a “Run Health Scan” feature into my Giga Pet project so I could demonstrate the different console logging methods inside Chrome DevTools in a more interactive and creative way instead of just printing random messages into the console. When the button is clicked, the application outputs different log levels including `console.log()`, `console.info()`, `console.warn()`, and `console.error()`. I also used `console.table()` to display Buddy’s stats in a cleaner format and `console.group()` to organize related debugging messages together. Lastly, I added a custom styled log message to show that console messages can also be visually customized.

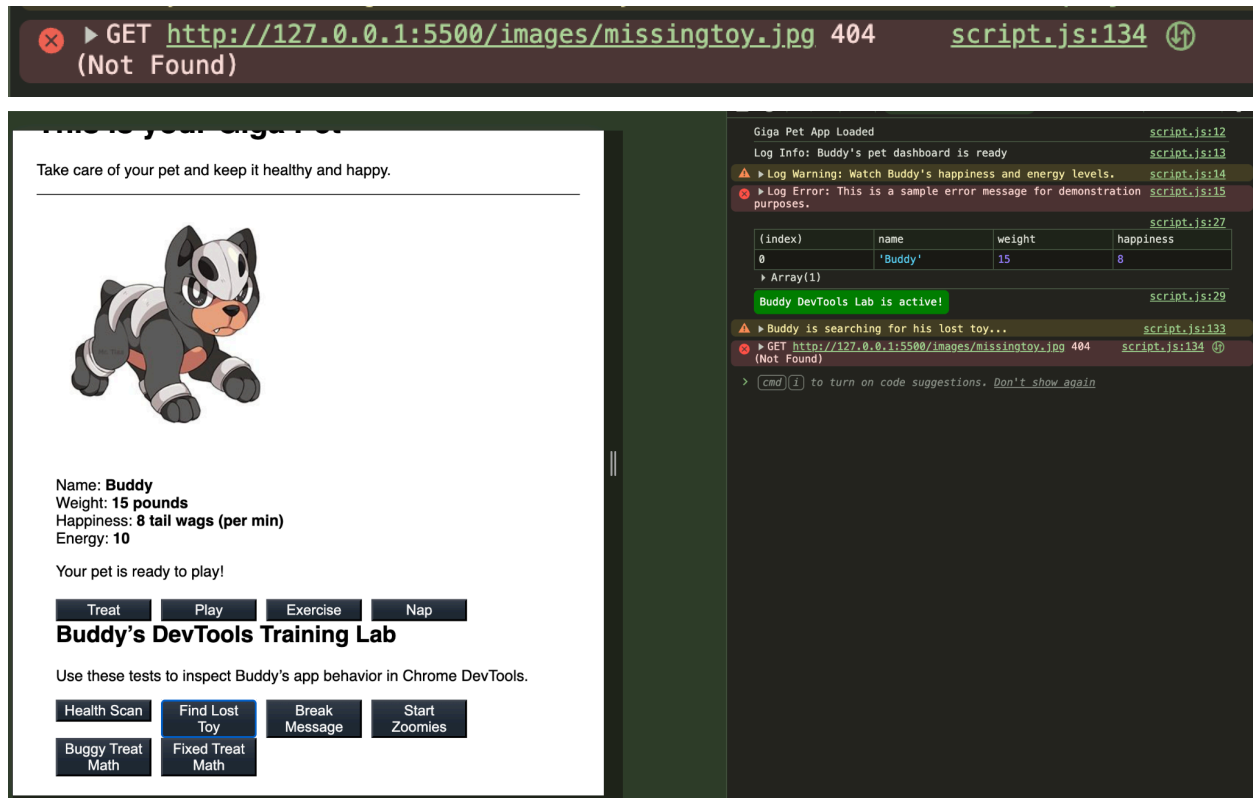
This implementation demonstrates how developers organize debugging information and filter messages depending on severity or importance.



2. 404 Network Error Simulation

For this implementation, I created a “Find Lost Toy” feature that intentionally tries to load a file that does not exist. I used the `fetch()` function to request a missing image file, which causes the browser to return a 404 “Not Found” network error inside Chrome DevTools.

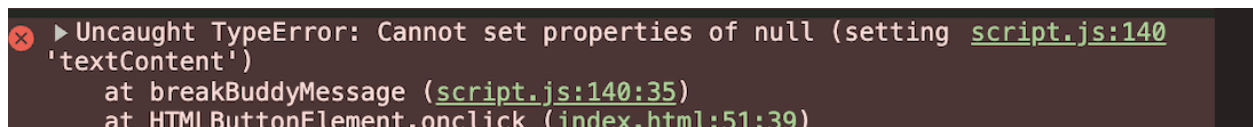
I wanted this implementation to simulate a realistic situation where a developer accidentally references a missing asset, image, or API endpoint.

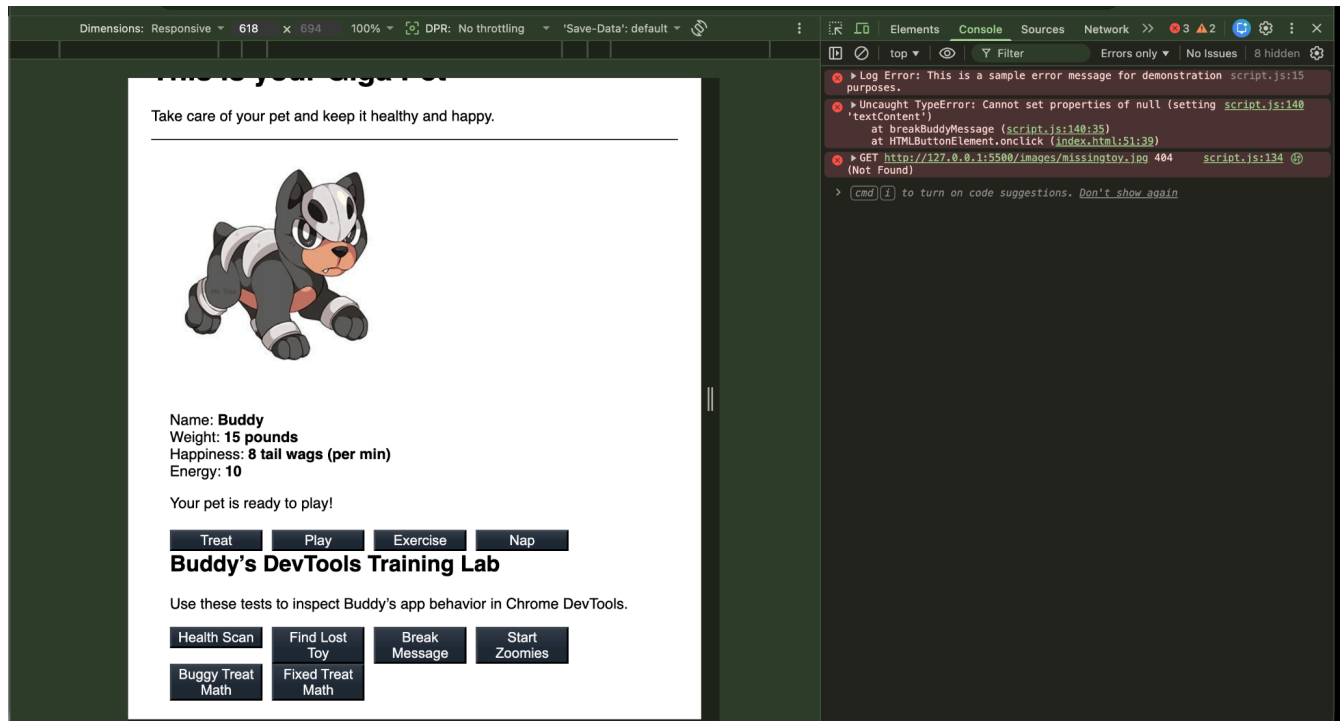


3. TypeError Simulation

For this implementation, I added a "Break Buddy Message" feature that intentionally causes a JavaScript TypeError. The function attempts to modify a null value, which causes JavaScript to throw an error because the object does not exist.

I added this because runtime errors are one of the most common problems developers debug using DevTools, and I wanted to demonstrate how Chrome identifies the exact location where the issue occurs.

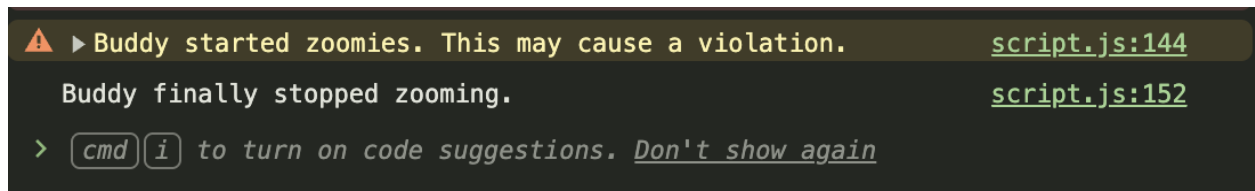




4. Performance Violation Simulation

For this implementation, I added a “Start Zoomies” feature that intentionally blocks the browser temporarily using a long-running loop. This causes Chrome DevTools to display a performance violation warning because the JavaScript execution takes too long.

I connected this idea to the Giga Pet theme by making it seem like Buddy is running around too much and overloading the application.

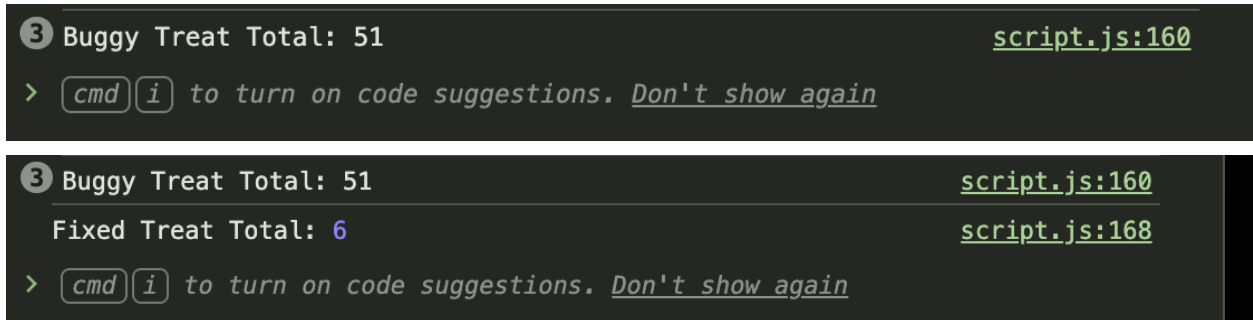


5. Bug Reproduction and Fix

For this implementation, I created a “Buggy Treat Calculator” feature that intentionally reproduces a logic bug in JavaScript. Instead of adding numbers correctly, the code combines two strings together and produces “51” instead of 6.

After reproducing the bug, I created a corrected version called “Fixed Treat Calculator” by converting the values into numbers before performing the addition.

I wanted to demonstrate the actual debugging process developers go through: reproducing the issue first, understanding why it happens, and then applying a fix.



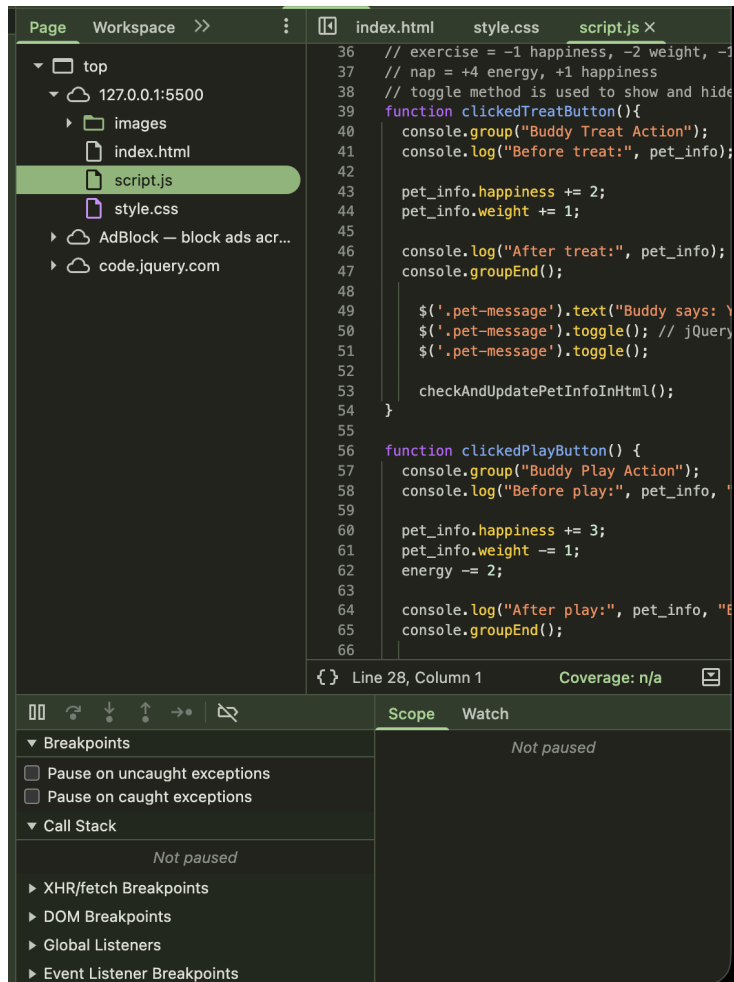
The image contains two screenshots of a code editor. The top screenshot shows a line of code: `3 Buggy Treat Total: 51` with a green link `script.js:160` to its right. Below the code is a prompt: `> [cmd] [i] to turn on code suggestions. Don't show again`. The bottom screenshot shows the same code, but with a second line of code added: `Fixed Treat Total: 6` with a green link `script.js:168` to its right. The prompt is also present below the code.

6. Sources UI and Breakpoint Debugging

For this implementation, I used the Chrome DevTools Sources tab to inspect my application's JavaScript file and set a breakpoint inside the Treat button function. Once the Treat button is clicked, execution pauses before the line runs, allowing me to inspect variables and application state in real time.

I also used the Scope pane and Watch Expressions to monitor values like Buddy's happiness and energy while the code was paused.

This implementation demonstrates how developers debug applications step-by-step and inspect what is happening internally during execution.



7. Console Filtering

For this implementation, I demonstrated Chrome DevTools filtering tools by filtering messages using log levels, text filtering, regular expressions, message sources, and user messages. This makes it easier to isolate important debugging information when the console becomes crowded.

I wanted to show that DevTools is not only useful for displaying messages, but also for organizing and analyzing them efficiently.

```
✖ ▶ Log Error: This is a sample error message for demonstration script.js:15
  purposes.

✖ ▶ Uncaught TypeError: Cannot set properties of null (setting script.js:140
  'textContent')
    at breakBuddyMessage (script.js:140:35)
    at HTMLButtonElement.onclick (index.html:51:39)

✖ ▶ GET http://127.0.0.1:5500/images/missingtoy.jpg 404 script.js:134 ⬇
  (Not Found)

> [cmd] [i] to turn on code suggestions. Don't show again
```

Dimensions: Responsive x 618 x 694 100% DPR: No throttling 'Save-Data': default

Elements Console Sources Network >> 3 2 8 hidden

top Filter Errors only No Issues 8 hidden

▶ Log Error: This is a sample error message for demonstration script.js:15 purposes.

▶ Uncaught TypeError: Cannot set properties of null (setting script.js:140 'textContent') at breakBuddyMessage (script.js:140:35) at HTMLButtonElement.onclick (index.html:51:39)

▶ GET http://127.0.0.1:5500/images/missingtoy.jpg 404 (Not Found) script.js:134

> [cmd] [i] to turn on code suggestions. Don't show again

Time to your dog's pet

Take care of your pet and keep it healthy and happy.



Name: **Buddy**
Weight: **15 pounds**
Happiness: **8 tail wags (per min)**
Energy: **10**

Your pet is ready to play!

Treat Play Exercise Nap

Buddy's DevTools Training Lab

Use these tests to inspect Buddy's app behavior in Chrome DevTools.

Health Scan Find Lost Toy Break Message Start Zoomies

Buggy Treat Math Fixed Treat Math